

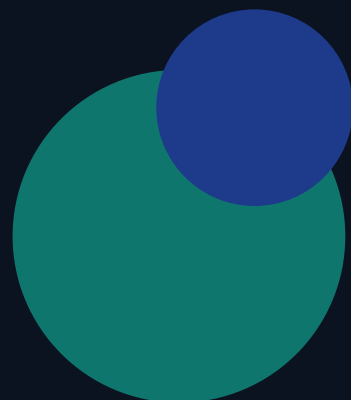
OWNER OPERATING GUIDE

Codex App Factory: Step-by-Step Guide

How to start a new personal app, prepare it for autonomous feature runs, operate the queue, review results, recover failures, and safely move from one app to the next.

Codex App Factory | Keshav Peri | 19 August 2026

Subscription-first. App-agnostic. Human-approved production.



Your operating model

You remain the product owner and production gate. Codex handles the repeatable analysis-build-test-review preparation loop. A new app becomes autonomous only after its product rules, commands, GitHub state, and smoke path are proven.

You own	The factory owns
Product intent and priorities	Selecting the next ready ticket
High-impact rules and privacy choices	Product-manager ticket gate
Accounts, billing, secrets, and production approval	Implementation and focused repair
Final review and merge	Independent QA and draft PR evidence

The full lifecycle

- 1 Workshop**
 Decide what the app is, its users, rules, data, non-goals, and V1 success.
- 2 Scaffold**
 Copy the generic harness and, when appropriate, a technology profile.
- 3 Wire**
 Create the app, GitHub repository, labels, real commands, CI, and deployment.
- 4 Smoke**
 Send one trivial visible ticket through the entire factory and owner merge path.
- 5 Queue**
 Write small features with observable definitions of done and label them ready.
- 6 Run**
 Trigger manually or on a conservative schedule, one feature per run.
- 7 Review**
 Inspect the draft PR, CI, QA evidence, and actual behavior; merge or clarify.
- 8 Improve**
 After real use, fix reusable definitions and add profiles only from evidence.

Phase 1 - Run the product workshop

Do not start autonomous building from an idea alone. The workshop should create a short, decisive product package - not a giant specification.

Required outputs

- Outcome and deadline: what must be usable, by whom, and when.
- Primary journey: the shortest successful end-to-end user flow.
- V1 features and explicit non-goals.
- Product and domain rules, including permissions, calculations, deadlines, and failure states.
- Data and privacy: what exists, where it lives, who can see or edit it, and how it is exported or deleted.
- Technology and deployment recommendation with reasons.
- Visual direction and important device or accessibility expectations.
- Definition of V1 done and an ordered initial feature list.

Before ticket writing

Resolve any decision that changes the data model, privacy boundary, identity model, hosting architecture, or core calculation. A feature ticket should not force the Builder to become the product owner.

What to bring back to the factory chat

- The completed workshop document or a clean summary.
- The ordered feature list.
- Any design references or mockups.
- Decisions that are still genuinely open.

Phase 2 - Choose a profile and scaffold

The factory has a generic core and optional technology profiles. Choose the smallest profile that is already justified by the product brief.

Choice	Use it when	Command
generic	The stack is not selected or no reusable profile fits.	<code>./scripts/new-app.sh /absolute/path/to/app</code>
static-web	A static npm web app with GitHub Pages is the deliberate architecture.	<code>./scripts/new-app.sh /absolute/path/to/app static-web</code>

```
cd /Users/keshav/Projects/codex-app-factory
./scripts/new-app.sh /Users/keshav/Projects/codex-apps/my-app
```

What the command copies

- Repository instructions and safety boundaries.
- Product Manager, Builder, and QA agent definitions.
- The orchestration skill and automation prompt.
- Product, design, decision, and project-configuration templates.
- The GitHub feature issue form and label setup helper.
- Profile-specific CI or deployment files only when a profile is selected.

Important

The scaffolder creates local files and initializes Git. It does not create accounts, choose product rules, create the remote repository, enable deployment, or merge anything.

Phase 3 - Fill the app's sources of truth

Before features enter the queue, replace every meaningful placeholder. These files are what a fresh Codex session will trust.

File	Complete it with
docs/product-brief.md	Outcome, users, V1 scope, non-goals, rules, data, privacy, architecture, and V1 done.
docs/design-reference.md	Personality, specific references, typography, colour, interactions, accessibility, and patterns to avoid.
docs/decisions.md	High-impact choices, reasons, rejected alternatives, and consequences.
factory/PROJECT-CONFIG.md	Actual install, test, lint, build, run, QA, preview, and production commands.

Project configuration rule

Agents must use the commands in PROJECT-CONFIG.md rather than guessing based on nearby files. If the stack changes, update the configuration and CI in the same change.

Quick completeness check

- Can another person explain the primary user journey?
- Are core calculations or domain rules unambiguous?
- Is every class of stored data named?
- Does the app have an export, backup, or recovery position where relevant?
- Are the real verification commands known?
- Are owner-only setup actions clearly named?

Phase 4 - Build the baseline and connect GitHub

The factory builds features inside a functioning repository. First establish a minimal app that installs, tests, builds, and starts locally.

1

Scaffold the application

Choose the framework or project type already approved in the product brief. Keep the baseline deliberately small.

2

Prove local commands

Run every setup, test, lint, build, and launch command recorded in PROJECT-CONFIG.md.

3

Commit main

Create a clean baseline commit before any autonomous feature branch.

4

Create the remote

Create a separate GitHub repository with the intended visibility and push main.

5

Create labels

Run scripts/configure-github.sh from the app repository.

6

Configure CI

Make the pull-request checks deterministic for this stack. Do not rely on agent judgment alone.

7

Configure deployment

Perform the owner-only provider setup named in the project configuration. Production should deploy only from reviewed main.

```
cd /Users/keshav/Projects/codex-apps/my-app
./scripts/configure-github.sh
```

Deployment boundary

The factory may include deployment code and verify configuration, but account creation, billing, secrets, and first-time production enablement remain owner-reviewed actions.

Phase 5 - Write a buildable backlog

A feature list becomes autonomous only after it is split into small GitHub issues with observable definitions of done.

A good ticket answers six questions

Question	Ticket field
What becomes possible for the user?	User-visible outcome
How will we know it works?	Definition of done
What may change?	In scope
What tempting adjacent work is excluded?	Out of scope
What must already exist or be decided?	Dependencies and decisions
Which devices, fixtures, or edge cases matter?	Verification notes

Ticket sizing

- One coherent user-visible outcome.
- One focused pull request.
- No unresolved high-impact product decision.
- A Builder can implement it and QA can verify it in one bounded run.
- If the description contains several independent 'and' clauses, split it.

Ready means reviewed

Apply factory:ready only when the issue is genuinely safe to build. The Product Manager still checks it, but the queue should not be a dumping ground for raw ideas.

Phase 6 - Prove the smoke path

Before scheduling real work, test the entire mechanism with a trivial visible change. The smoke ticket tests the factory, not the product.

1

Create a trivial issue

For example: add a small version label or an unmistakable placeholder message. Include complete acceptance criteria.

2

Label it ready

Confirm it is the oldest open factory:ready issue.

3

Run manually

Open the app repository in Codex and invoke the factory skill.

4

Watch the handoffs

Confirm Product Manager -> Builder -> QA -> orchestrator is visible and responsibilities remain separate.

5

Inspect the draft PR

Require outcome, issue link, preview or local evidence, verification, decisions and risks, and owner review steps.

6

Verify CI

All required deterministic checks must pass.

7

Use the change

Exercise the actual behavior locally or through a real preview.

8

Merge yourself

Confirm production changes only after your merge.

9

Test the empty queue

Run again with no ready issue and confirm a cheap 'Nothing ready' exit.

Fix the definition, not the demo

If the smoke run exposes a systemic problem, correct the shared agent, skill, template, or project configuration and repeat the smoke test. Do not hand-edit a single successful-looking output and declare the factory healthy.

Phase 7 - Run one feature manually

Manual runs are the normal starting point and remain useful for urgent work or troubleshooting.

Start the run

```
Open the app repository as the Codex project.  
Prompt: Use $codex-app-factory to process the next ready feature.
```

What you should see

- A quick GitHub queue check before broad exploration.
- At most one ticket selected.
- A product gate before code changes.
- One Builder writing code.
- QA independently reviewing the diff and behavior.
- No more than two focused Builder revisions.
- A draft pull request, never an autonomous merge.

If the issue blocks

Read the single question on the issue. Answer it, update the brief or ticket if the answer changes durable product context, and return the label to factory:ready. Do not merely tell a chat the answer if the next run could not discover it.

Recoverable branch rule

If a run stops after code changes, keep the branch. The next run should recover work for that issue number instead of creating a second implementation.

Phase 8 - Configure scheduled runs

Schedule only after the manual smoke path is trustworthy. The scheduled task belongs to the app repository, not the factory repository.

1

Open scheduled tasks

In the Codex desktop app, create a scheduled task for the target app project.

2

Choose the working mode

Use the local project or an isolated worktree. Worktrees reduce interference with your interactive checkout.

3

Use the canonical prompt

Paste the exact contents of `factory/AUTOMATION-PROMPT.md`.

4

Start conservatively

Run once per chosen work period and keep the one-ticket cap. Increase only after observing usage and reliability.

5

Keep the machine available

Local scheduled tasks need the computer awake and the Codex desktop app running.

6

Check the scheduled inbox

Review each run's outcome, branch, blocked question, or draft PR.

Canonical prompt

```
Use $codex-app-factory to process at most one ready feature in this repository.
Begin with the cheap queue check. Never merge or perform an owner-only action.
```

Not an always-on server

If the Mac is asleep, a local scheduled task cannot work on the repository. Moving to a hosted or API-backed runner later would require a new cost, security, and architecture review.

Phase 9 - Review and merge

The draft pull request is the handoff from the autonomous team to you. Review behavior, not just prose.

Five-minute review

- 1 Outcome**
Can you explain the user-visible change in one sentence, and does it match the issue?
- 2 Evidence**
Did CI pass? Did QA actually run the relevant checks? Is any limitation stated honestly?
- 3 Behavior**
Use the feature through the shortest reliable manual path supplied in the PR.
- 4 Scope**
Look for adjacent behavior, dependencies, or infrastructure that the ticket did not authorize.
- 5 Risk**
Read migrations, data impact, privacy, accessibility, and follow-up notes.
- 6 Decision**
Merge only when you understand and accept the result. Otherwise comment with observed behavior and revise the ticket.

Merge	Send back
Acceptance checks are visibly satisfied	The feature behaves differently from the ticket
CI and QA pass	Evidence is missing, guessed, or contradicted
No unexplained owner-only action	The change widens cost, privacy, or authority
You can explain the change	The PR is too broad to review confidently

Phase 10 - Operate the queue

Once the factory is healthy, your recurring job is product ownership: maintain a small, high-quality ready queue and review completed work.

Daily or run-day

Check blocked questions, inspect draft PRs, use the changed feature, merge or clarify, and keep no more ready work than you can review.

Weekly

Review usage, repeated failures, flaky tests, stale branches, outdated product rules, and whether the schedule still matches your capacity.

Backlog hygiene

- Keep raw ideas unlabelled or outside the ready queue.
- Prioritize by issue order; do not create several competing sources of priority.
- Close or rewrite stale tickets when the product brief changes.
- Put reusable corrections into the factory only when they apply across apps.
- Keep app-specific rules inside the app repository.

Capacity rule

Autonomy does not remove review responsibility. A growing factory:review queue means the schedule is too aggressive for the owner gate.

Failure playbook

Symptom	First action	Do not
Nothing ran	Check schedule, target project, Mac availability, and Codex app state.	Create duplicate tasks immediately.
Nothing ready	This is a successful cheap exit; queue a reviewed issue when desired.	Treat it as a failure.
Ticket blocked	Answer the one question in durable issue or brief state, then relabel ready.	Answer only in an unrelated chat.
Run died during build	Inspect the issue branch and recover it by issue number.	Start a second branch blindly.
Tests fail twice	Split or re-specify the ticket and preserve the evidence.	Allow an endless repair loop.
No preview	Use honest local QA evidence or deliberately add preview infrastructure.	Invent a URL or claim unperformed QA.
Usage too high	Reduce frequency, scope, context, and optional parallelism.	Remove the product or QA gates first.
Wrong product behavior	Correct the brief or ticket, then rebuild from the clarified rule.	Patch code while leaving the source of truth wrong.

Systemic failure rule

When several apps could suffer the same failure, improve the canonical factory and re-copy or port the fix deliberately.
When only one product is affected, fix that app's context or code.

Finishing one app and starting another

A personal app is ready for steady operation when its intended V1 journey works, its ready queue contains no required V1 tickets, and any omissions are consciously recorded.

End-of-V1 checklist

- Definition of V1 done is met through actual use.
- Required tests and deployment are stable.
- Data export, backup, recovery, or deletion behavior is known where relevant.
- No required V1 issue remains in factory:ready.
- Known limitations are written, not merely remembered.
- The operating schedule is appropriate for the remaining backlog.

Retrospective before app number two

- Which tickets blocked, and were their specifications weak?
- Which QA findings repeated?
- Which project commands or evidence were missing?
- Which factory rule genuinely applies to future apps?
- Should a new technology profile be created from repeated stack-specific setup?

App-agnostic discipline

Do not turn one app's product rules into factory law. Generalize orchestration, safety, recovery, and reusable stack setup - not the domain itself.

Quick reference

Commands

```
# Create a generic app
./scripts/new-app.sh /absolute/path/to/app

# Create a static web app
./scripts/new-app.sh /absolute/path/to/app static-web

# Configure factory labels after GitHub remote exists
./scripts/configure-github.sh

# Manual factory prompt
Use $codex-app-factory to process the next ready feature.
```

Non-negotiable rules

- One ready feature per run.
- Product gate before implementation.
- One code writer per working tree.
- Independent QA after implementation.
- No more than two Builder revisions.
- Durable state in GitHub and repository files.
- No autonomous production merge.
- No account, billing, secret, destructive data, or privacy-boundary action without explicit approval.

Official Codex references

- AGENTS.md: <https://learn.chatgpt.com/docs/agent-configuration/agents-md>
- Subagents and custom agents: <https://learn.chatgpt.com/docs/agent-configuration/subagents>
- Skills: <https://learn.chatgpt.com/docs/build-skills>
- Scheduled tasks: <https://learn.chatgpt.com/docs/automations?surface=app>

When in doubt

Preserve state, stop before widening authority, ask the smallest useful question, and make the answer discoverable to the next run.